P O R T F O L I O

정연서

Software Engineer

010-6632-0568 · github.com/yeonseo-Jung · linkedin.com/in/yeonseo-jung

01

분산 트레이딩 시스템 Zero-to-One 구축 + Rust 마이그레이션

pub/sub 1:N 브로드캐스트 · 24/7 무중단 · Python→Rust 무중단 전환 (메모리 -95%)

02

quantus-pipeline 레거시 Airflow 현대화

3년간 유지보수한 레거시를 단독 재설계·전환

03

퀀터스 프라임 글로벌 올웨더 액티브 전략

가설 수립 · 피처 엔지니어링 · 운용 성과 (Sharpe 1.5 · 단일 전략 AUM 50억)

PROJECT 01

분산 트레이딩 시스템 — Zero-to-One 구축 + Rust 마이그레이션

STOA Investment · 2023 – 현재 · 설계·구현·운영·고도화 단독 수행

Python | Rust | RabbitMQ | Celery | tokio | sqlx | XGBoost · LightGBM | pytest

---

PHASE 1

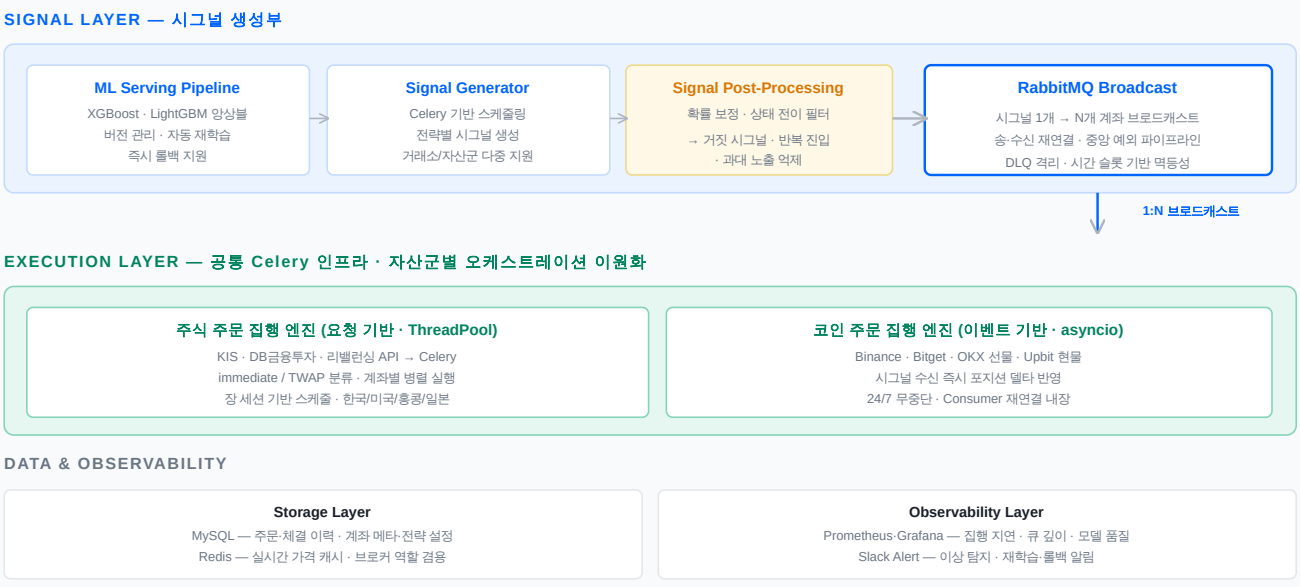
Python 기반 pub/sub 1:N 분산 구조 Zero-to-One 설계

2023 – 2025 · 1,600+ 계좌 · 24/7 무중단 운영 진입

프로젝트 배경

계좌 수가 수백을 넘어서면서 기존 단일 프로세스 구조로는 감당이 안 되는 문제가 드러났습니다. 시그널 생성과 주문 집행이 같은 프로세스에 묶여 있어 한쪽 문제가 전체로 번졌고, **24/7 이벤트 매매가 필요한 코인**과 **장 세션·리밸런싱 기반으로 돌아가는 주식**을 하나의 엔진으로 처리하기도 어려웠습니다. 기존 시스템을 확장하는 대신 **Zero-to-One으로 pub/sub 기반 분산 구조를 새로 설계**했고, 현재 1,600+ 계좌 · 6개 외부 시스템을 **24/7 무중단 · 메시지 유실 0건**으로 운용 중이며, 월평균 거래대금은 약 5천억 규모입니다.

아키텍처 — pub/sub 1:N 분산 구조



*시그널 생성부 → RabbitMQ → 주문 집행부로 흐르는 1:N 분산 구조 · 코인은 이벤트 기반 신호 매매, 주식은 요청 기반 TWAP 오케스트레이션으로 **트리거 방식과 오케스트레이션 패턴을 이원화** · 하단은 데이터 저장 계층과 관측성 계층으로 분리*

핵심 설계 판단

PUB/SUB 기반 1:N 분산 구조 — 메시지 브로커 선택

시그널 생성과 주문 집행을 물리적으로 분리하고, **같은 시그널을 수많은 계좌에 동시 전달하되 메시지 유실을 0으로 유지**하는 것이 핵심 요구였습니다. 소규모 팀 운영 부담까지 고려해 RabbitMQ·Kafka·NATS JetStream·Redis Pub/Sub 4개 대안을 비교한 결과, **"1:N 브로드캐스트 네이티브 + 메시지 보장 + 낮은 운영 부담"의 교집합이 가장 넓은 RabbitMQ를 선택**했습니다. 상세 비교와 재평가 조건은 아래 별도 섹션으로 분리했습니다.

메시지 유실 방지는 **송신·수신·처리 세 층위**에서 설계했습니다. **송신 측**은 Publisher 재연결과 중앙 예외 파이프라인을 통해 장애 시 알림·로깅·재처리를 일원화하고, **수신 측**은 Consumer 재연결과 DLQ 격리로 포이즌 메시지가 전체 흐름을 블로킹하지 않도록 분리했습니다. at-least-once 보장은 필연적으로 중복 수신 가능성을 내포하므로, **처리 측은 시간 슬롯 단위 캐시 키 기반 멱등성**으로 동일 시그널의 중복 처리를 차단합니다.

주식·코인 오케스트레이션 이원화 (공통 실행 인프라 + 분기된 트리거 방식)

주식과 코인은 **동일한 Celery 기반 실행 인프라를 공유**하되, 시그널 공급 방식과 오케스트레이션 패턴을 분기했습니다. **코인**은 RabbitMQ 시그널을 수신하면 즉시 포지션 델타를 반영하는 **이벤트 기반 신호 매매**로, **주식**은 리밸런싱 API 호출로 트리거되어 immediate/TWAP 분류 후 계좌별 ThreadPool로 병렬 집행하는 **요청 기반 TWAP 오케스트레이션**으로 구성했습니다. 24/7인 코인과 장 세션·휴장일을 의식해야 하는 주식은 체결 요구 조건이 본질적으로 다르기 때문에, 공통 인프라를 유지하면서도 **트리거 모델을 이원화하는 것이 억지 통합보다 운영·확장 양 측면에서 유리**하다는 판단이었습니다.

ML 서빙을 시그널 파이프라인의 일부로 통합

모델 예측과 후처리를 **별도 서비스가 아니라 시그널 생성부의 내부 단계**로 두어 네트워크 홉과 운영 포인트를 줄였습니다. 모델 버전은 **재현·이식·롤백이 쉬운 포맷으로 저장**해 환경이 바뀌어도 동일 모델을 그대로 복원할 수 있도록 했고, 주기적 자동 재학습과 실패 시 즉시 이전 버전으로 되돌리는 구조를 기본 운영 루틴에 넣었습니다.

메시지 브로커 대안 비교 · 마이그레이션 트리거

대안	1:N 브로드캐스트	메시지 보장	운영 부담	판단
RabbitMQ	Fanout 네이티브	at-least-once + ACK	낮음 (단일 인스턴스로 시작 가능)	**채택**
Kafka	컨슈머 그룹·파티션 모델로 재구성 필요 (Fanout exchange 대비 모델 mismatch)	강함 (replay 가능)	높음 (3+ broker 클러스터)	현 규모에 과잉
NATS JetStream	네이티브 지원	at-least-once	낮음 (단일 바이너리, 경량)	유력 대안 (Python 생태계 성숙도 관찰)
Redis Pub/Sub	지원	at-most-once (유실 가능)	이미 스택에 존재	금융 도메인 부적합

향후 마이그레이션 트리거 — 현 선택은 지금 규모·요구에 최적이지만 다음 조건에서 재평가합니다:

- **계좌 수·메시지 레이트 10배 이상 확장** + ms 지연이 병목으로 드러날 때 → **NATS JetStream**으로 점진 이관 검토 (pub/sub 모델이 유사해 전환 비용 낮음)
- **이벤트 재생·감사 로그·과거 시점 재현**이 핵심 요구가 될 때 → **Kafka** 전환 검토 (로그 중심 아키텍처 재설계 동반, 비용 높음)
- 그 외에는 **현 선택 유지**. 운영 부담·안정성 측면에서 RabbitMQ가 여전히 유리

금융 도메인 신뢰성 설계 포인트

주문 체결은 실패 시 실제 자산 손실로 직결되는 도메인이라, 메시지 레벨의 유실 방지 외에도 **주문 단위·계좌 단위·운영 단위 각 층위에서 추가 안전장치**를 설계했습니다.

→ **TWAP 과체결 방지 3중 방어** — 라운드 시작 시 ① 미체결 주문 취소 → ② 보유 수량 재조회로 실 상태 재계산 → ③ 최종 목표 수량으로 주문 규모 클램핑. 장중 포지션이 흔들려도 누적 주문이 목표를 초과하지 않음

→ **장 마감 기준 라운드 매 틱 재산정** — 총 주문 시간/주문 간격으로 기본 라운드를 산정하되, **장 마감까지 남은 시간을 기준으로 매 틱 재계산**해 장 종료 이후 주문이 발생하지 않도록 차단

→ **오류 코드 분류 기반 재시도** — 거래소 오류를 *무시 가능 / 재시도 가능*으로 분류해 라운드 루프의 안정성 확보. 포괄적 예외 처리로 인한 "실패 원인 불명" 상황 차단

→ **계좌 자동 일시중단 (Circuit Breaker · Bulkhead 효과)** — 특정 계좌에서 거래소 오류가 반복되면 해당 계좌를 자동 비활성화(Circuit Breaker open). 한 계좌의 문제가 **전체 집행 루프로 연쇄 확산되는 것을 격벽으로 차단**하는 Bulkhead 효과를 부수적으로 확보

→ **시간 슬롯 단위 시그널 중복 방지** — at-least-once 메시징의 필연적 중복 수신을, 시간 슬롯 기반 캐시 키로 **동일 슬롯의 재처리를 차단**. 금융 도메인에서 중복 주문은 곧 손실이므로 이 층위가 필수

→ **중앙 예외 파이프라인** — 예외 로깅·Slack 알림·DB 기록을 단일 지점으로 통합. 개별 태스크에 흩어진 에러 처리 대신 **관찰 가능성과 원인 분석을 일원화**

→ **ML 서빙 품질 자동 감시** — **학습→예측→재학습 생애주기 관리**와 **3계층 검증 체계**(합성 데이터 유닛 테스트 · 실데이터 기반 통합 검증 · 정량 보고서)로 모델 품질을 자동 모니터링. 모델 오염이 실 주문에 영향을 주지 않도록 사전 차단

성과

1,600+ 계좌 | **월평균 거래대금 약 5천억** | 24/7 무중단 운영 | 6개 거래소/증권사 통합 | 메시지 유실 0건

Binance · Bitget · OKX 선물 + Upbit 현물 + KIS · DB금융투자 주식을 단일 아키텍처로 통합 운용 중입니다. 시그널 생성부·주문 집행부 분리로 부분 장애가 전체를 중단시키지 않으며, 수평 확장이 계좌 단위·거래소 단위로 가능해졌습니다. **"메시지 유실 0건"**은 publisher confirms로 broker ACK 검증 + manual ACK consumer + **at-least-**

once + 시간 슬롯 멱등성으로 보장하는 형태이며 (exactly-once의 강한 주장은 아닙니다), DLQ에 적재된 메시지는 운영 중 모두 재처리해 손실 없이 회수해온 결과입니다.

> **이 설계가 일반화되는 도메인**
>
> 1:N 메시지 브로드캐스트, at-least-once + 시간 슬롯 멱등성, DLQ 격리, Circuit Breaker 기반 격리는 **거래 시스템에 한정된 기법이 아닙니다**. 채팅·알림·실시간 동기화처럼 메시지를 다수 대상에 안정적으로 전달해야 하는 모든 시스템에서 동일한 신뢰성 요구를 만납니다. 이 시스템에서의 설계 판단은 **도메인이 바뀌어도 그대로 옮겨갈 수 있도록 트레이드오프 형태로 분리**해 두었으며, 위 메시지 브로커 대안 비교 표와 신뢰성 설계 포인트가 그 형태입니다.

현재 남은 한계와 부채

현재 시스템의 주요 한계는 세 가지이며, 각각 **해결 조건과 재평가 시점을 함께 관리**하고 있습니다. 첫째, at-least-once 메시징 기반이라 **이벤트 재생·과거 시점 재현이 불가능**해, 사후 분석이나 감사(audit) 요구가 커지면 별도 이벤트 스토어 도입이 필요합니다(대안 비교 테이블의 Kafka 전환 트리거와 연결). 둘째, 스케줄러·브로커의 **HA 구성은 현 규모의 비용 효율이 맞지 않아 도입하지 않았고**, 월평균 거래대금이 현재의 5배 이상으로 확장되거나 다운타임 허용치가 강해질 때 재평가 대상입니다. 셋째, TWAP 파라미터(주문 시간·간격)는 계좌별 설정은 가능하나 **시장 상황에 따른 동적 조정 로직이 없어**, 고변동성 국면에서의 체결 품질을 실측 기반으로 조정하는 로직이 중기 과제입니다.

PHASE 2

Python → Rust 거래 실행 엔진 마이그레이션

2025 – 2026 · Strangler Fig + Feature Flag 기반 무중단 전환 · 단독 수행

Rust | tokio | sqlx | lapin | axum | Cargo workspace | 3-Stage Docker | GitHub Actions

마이그레이션 동기 — 단순 성능 개선이 아닌 구조적 한계

Phase 1에서 구축한 Python 시스템을 1,000+ 계좌 규모로 운영하면서, 단순 성능 문제가 아니라 **동일 패턴의 프로덕션 장애가 반복 표출**되는 것이 핵심 동기였습니다. GIL·이벤트 루프·드라이버 호환성·worker 블로킹 등은 각각 별도 사고가 아니라 **"타입 시스템 부재"와 "async 런타임 일관성 부재"라는 하나의 근본 원인**으로 수렴했고, 라이브러리·설정 패치로는 재발을 막을 수 없는 영역이라고 판단했습니다. 언어 수준 해결이 필요하다는 결론은 이 인식 위에서 내려진 것입니다.

PYTHON 시스템의 구조적 한계	표면 증상	근본 원인
GIL 병목	계좌 단위 동시 실행을 `asyncio.Semaphore(50)` 로 배치 분할	거래소 API 호출 전 **서명 생성·요청 직렬화 워크플로(CPU-bound)**가 GIL로 직렬화
Celery prefork × asyncio 충돌	`"attached to a different loop"` + 전체 태스크 실패	태스크마다 `asyncio.run()` → 이벤트 루프 반복 생성·소멸 → 풀링 포기(NullPool)
TWAP `time.sleep()` 블로킹	최대 6.5h worker 점유 + 처리량 급락	동기 sleep이 worker를 직접 점유 → `prefetch_multiplier=1` 과 결합 시 자원 회수 불가
RabbitMQ `consumer_timeout` 중복 체결	동일 주문이 2회 이상 체결	장시간 태스크가 컨슈머 연결을 점유 → 타임아웃 → 메시지 재전달 → 중복 실행 (Celery task 생명주기와 ACK이 결합된 구조)
동기↔비동기 드라이버 호환성	전체 DB 연결 실패 (`asyncmy` `write_timeout` 사건)	같은 MySQL 프로토콜이라도 pymysql/asyncmy 파라미터 호환성 보장 없음 — 덕 타이핑이 런타임까지 은닉

언어 선택 — 대안 비교 및 의사결정

탈락안 1 · GO — 인터페이스 계약의 명시적 표현력에서 차이

4개 거래소 어댑터의 공통 계약을 단일 인터페이스로 통일해야 하는 요구가 핵심이었습니다. Go의 **structural typing(암묵적 인터페이스 충족)**은 유연성 측면에서 장점이지만, 거래소가 N개로 확장될 때 **새 어댑터가 계약을 부분 충족할 경우 실패가 다른 호출 지점에서 표면화**되어 디버깅 표면적이 넓어집니다. Rust trait은 `impl Trait for T` 를 명시적으로 강제하고 trait bound로 함수 시그니처에 제약이 박혀, 거래소 추가 시 **"무엇을 구현해야 하는지"가 컴파일러에 의해 정확히 안내**됩니다. 제네릭 성숙도(1.18+) 도 trait bound 수준의 제약 표현에는 미흡했습니다.

탈락안 2 · JAVA/KOTLIN — JVM 런타임 비용

JVM 콜드 스타트(~2-5초) + JRE 포함 Docker 이미지 비대(~300MB+) 가 본 마이그레이션의 핵심 목표인 **"메모리 효율로 계좌 확장 여력 확보"**와 정면 충돌했습니다. Celery worker × 36의 메모리 절감 효과가 제한적입니다.

탈락안 3 · C++ — 메모리 안전성 부재

금융 시스템에서 use-after-free·buffer overflow는 자산 손실로 직결됩니다. 이 도메인에서 메모리 안전성을 컴파일러가 보장하지 못한다면 도입 자체가 위험을 늘리는 결정입니다. 비동기 생태계(Boost.Asio) 성숙도도 tokio 대비 부족했습니다.

선택 · RUST — 위 한계의 동시 충족

① **GIL 없는 진정한 병렬성** (tokio M:N 스케줄링· `JoinSet` 으로 1,000+ 계좌 전체 동시 spawn) · ② **Zero-cost abstraction** (제네릭 + trait bound·match 정적 디스패치 — 추상화에 런타임 비용 0) · ③ **컴파일 타임 안전성** (소유권 시스템 + 정적 타입 → 드라이버 호환성· `**kwargs` 시그니처 누락 같은 장애를 컴파일 시점에 차단) · ④ **GC 부재 + 단일 바이너리** (Celery worker × 36 대비 메모리 절감) · ⑤ **async 생태계 성숙도** (tokio + axum + sqlx + lapin + reqwest — 모두 프로덕션 검증 완료).

마이그레이션 전략 — Strangler Fig + Feature Flag

1,000+ 계좌가 실시간 운용 중인 시스템이라 **단 한 번의 배포 실패도 허용되지 않는** 전제에서 출발했습니다. Big Bang 전환을 배제하고, Python 서비스와 Rust 서비스가 RabbitMQ를 경계로 공존하며 **환경변수 하나로 즉시 전환·롤백**되는 구조를 채택했습니다.

참고로 Python 생태계 내 대안(`arq`·`taskiq` 등 async-native 태스크 큐)으로 Celery 한계를 완화하는 경로도 검토했으나, ① 운영 중 시스템의 마이그레이션 비용이 동등 시간의 Rust 전환 비용에 가깝고 ② 언어 수준 전환은 위 한계 패턴의 **재발 자체가 구조적으로 불가능**해진다는 점에서 영구 해결 효과가 더 컸기 때문에 후자를 선택했습니다.

3-MODE FEATURE FLAG · 동시 소비 차단

`QUANTUS_ENGINE` 환경변수 하나로 운영 엔진을 전환합니다. `python` 모드에서는 Rust executor가 즉시 return, `rust` 모드에서는 Python consumer가 즉시 return하여 **동시 소비로 인한 중복 주문을 코드 레벨에서 구조적으로 차단**합니다. `shadow` 모드에서는 Rust가 주문 수량만 계산하고 실주문은 전송하지 않아 양 엔진 결과를 무위험 비교할 수 있도록 설계했습니다. 전환 단위는 환경변수이므로 **L1 롤백(Feature Flag)은 5초 이내**이며, L2(이미지 롤백 ~20초)·L3(히스토리 복원)의 3계층 롤백으로 운영합니다.

불변 원칙 · 사이드 이펙트 차단

기존 DB 스키마·RabbitMQ exchange 이름·Redis 키 네임스페이스·기존 API 응답 포맷은 **변경하지 않는다**는 규칙을 마이그레이션 기간 내내 강제했습니다. 새 구조가 필요하면 **새 키·새 컬럼·새 테이블만 추가**하고 기존을 보존했습니다. 이는 페이즈별 즉시 롤백 가능성과 직결됩니다.

타깃 아키텍처 — 13-crate Cargo Workspace

관심사 분리와 순환 참조 방지를 위해 **3-tier 의존성 구조**로 설계했습니다. 거래소별 어댑터는 독립 크레이트로 분리하여 추가 거래소가 다른 영역에 영향을 주지 않도록 격리했습니다.

```
quantus-rs/  (13 crates · 11,461 LOC)
│
├─ Tier 1 · Foundation
│   ├─ quantus-common      공유 타입 · 설정 · 에러
│   ├─ quantus-db          sqlx MySQL 비동기 풀
│   └─ quantus-messaging   RabbitMQ (lapin)
│
├─ Tier 2 · Domain
│   ├─ quantus-signal      시그널 생성 (ndarray)
│   ├─ quantus-risk        리스크 관리
│   ├─ quantus-data        가격 수집·캐싱
│   └─ exchange-adapters/  거래소 독립 크레이트
│       ├─ binance-rs      HMAC-SHA256
│       ├─ bitget-rs       HMAC-SHA256 + passphrase
│       ├─ okx-rs          Base64 + HMAC + ISO8601
│       └─ upbit-rs        JWT
│
└─ Tier 3 · Application
    ├─ quantus-executor    거래 실행 오케스트레이션 (핵심)
    ├─ quantus-scheduler   tokio-cron-scheduler
    └─ quantus-api         axum + 헬스체크
```

거래소 어댑터 · TRAIT 기반 인터페이스 계약

거래소별 차이(심볼 변환·서명 알고리즘·배치 주문 한도)는 어댑터 내부에 캡슐화하고, 외부에는 `ExchangeAdapter` trait이 단일 계약을 강제합니다. 거래소별 초기화 파라미터가 상이하므로(Bitget passphrase, OKX ISO8601 서명, Upbit JWT 등) **match 기반 정적 디스패치**로 시작했고, `Send + Sync` trait bound는 향후 동적 디스패치(`Box<dyn ExchangeAdapter>`) 전환 가능성을 열어두었습니다. 거래소 추가는 **새 크레이트 + 팩토리 등록**만으로 가능하도록 구조화했습니다.

운영 결과 — 검증된 정량 지표

메모리 4.3GB → 193MB (-95%) | **Docker 1.2GB → 160MB (-87%)** | 트레이딩 핵심 LOC -53%

4개 거래소 25개 전략 큐 단독 운영 | Worker 블로킹 제거 (TWAP non-blocking)

Python Celery worker × 36으로 분산 실행하던 **코인 거래 실행 엔진을 Rust 단일 바이너리로 대체**해 운영 중입니다. 25개 전략 큐를 RabbitMQ에서 직접 구독하며, `QUANTUS_ENGINE=rust` 전환 후 Python consumer는 자동 비활성화됩니다. 메모리 절감분은 향후 계좌 확장 여력으로 직접 환산되는 자산입니다.

검증된 효과 VS 미측정 항목 — 솔직한 한계

마이그레이션의 효과는 ① **자원 효율**(메모리·이미지·LOC 실측) ② **특정 장애 클래스의 구조적 소멸**(아래 표) 에서 확실히 검증되었습니다. 다만 **지연(latency)의 before/after 정량 비교는 미수집**입니다 — 전환 시점에서 **"Shadow 파이프라인 완성 대기" vs "조기 전환 + 실거래 관찰"** 의 트레이드오프를 따져, Python 시스템에서 동일 패턴 장애가 지속 발생 중이었고 Feature Flag 5초 롤백이 가능했기에 **의식적으로 후자를 선택**했습니다. 그 결과 전환 직후 hotfix 14건(1차 10건은 Shadow 사전 검출, 2차 4건은 실거래 관찰로 수정)이 발생했고, 이 비용은 사전에 예상한 범위 내였습니다. 이 경험은 **"Shadow 사전 검출 + 실거래 관찰의 병용이 최소 위험 경로"**라는 교훈으로 정리했고, 동등 상황 재현 시 Shadow 완성 우선 검토 가능성을 함께 기록해 후속 과제로 관리 중입니다.

학습 곡선과 적용 — 신규 언어 도입 방법론

Rust는 본 마이그레이션 시점에 **본격적으로 처음 다룬 언어**였습니다. 시스템 엔지니어링·분산 시스템·금융 도메인은 깊은 자산이었으므로, **학습 대상은 언어 고유 영역(소유권·라이프타임·`async` / `Send` / `Sync` ·trait object·`Pin` / `Future`)**에 집중되었고, 풀어야 할 문제 자체는 이미 익숙한 영역이었습니다. 운영 중 시스템에 직접 적용하기 위해 **학습 → 격리된 설계(Strangler Fig + Feature Flag) → Shadow Mode 검증 경로 → 점진 전환**의 시간 순서를 강제했고, **위험을 "언어 미숙"이 아니라 "전환 전략"으로 분리**했습니다. 새로운 도구의 학습이 곧 운영 적용으로 이어진 사례이며, 채택할 도구가 변할수록 가치가 커지는 형태의 역량으로 본인의 강점을 자리매김하고 있습니다.

영구 소멸한 장애 클래스

Phase 1에서 반복 재발하던 패턴이 Rust로 전환된 영역에서는 **언어·런타임 레벨에서 재발 불가능**해졌습니다. 이는 라이브러리 패치가 아닌 구조적 해결입니다.

PYTHON에서의 문제	RUST의 구조적 해결
동기↔비동기 드라이버 파라미터 호환성 (`asyncmy` `write_timeout` 류)	단일 드라이버(sqlx), 컴파일 타임 검증
이벤트 루프 생명주기 충돌 (`"attached to a different loop"`)	tokio 단일 런타임, 앱 생명주기와 동일
`**kwargs` 프록시 체인 시그니처 누락 (`use_nxt` 사건)	정적 타입 시스템, 동적 전파 패턴 자체가 불가
Worker 블로킹 (`time.sleep()` 자원 점유)	`tokio::time::sleep()` non-blocking
메시지 브로커 timeout 중복 체결	즉시 ACK + 별도 tokio 태스크 (실패 시 DB 재시도 큐로 재투입)
DB 스키마 타입 불일치를 덕 타이핑이 은닉	기동 즉시 타입 불일치가 표면화 — 잠재 결함의 가시화

역설적 교훈 · MySQL JSON 컬럼 사건 — Rust 기동 직후 crash loop 발생. DB의 `leverage_assets` 등이 MySQL JSON 타입이었지만 Rust 모델이 `Option<String>` 으로 선언되어 sqlx가 자동 변환을 거부한 사건입니다. Python 덕 타이핑이 **오랫동안 감춰둔 스키마 불일치**가 정적 타입 시스템에서 즉시 표면화된 것 — 이는 마이그레이션의 가치를 역설적으로 증명한 사례로 기록했습니다.

의도된 Python 잔존 영역

한국 주식(KIS·DBFI) 거래는 외부 SDK(Pykis·Pydb)가 Python 전용이고, Rust 네이티브 포팅은 6개월+의 검증 기간이 필요한 작업입니다. **금전적 위험 대비 ROI를 검토한 결과, Python 서비스 유지(Option A)를 의식적으로 선택**했고, 시그널 생성 엔진과 주식 TWAP 역시 동일 판단으로 잔존시켰습니다. **"전부 Rust로 옮기지 않은 것"은 마이그레이션의 미완이 아니라 의도된 결정**이며, 코인 거래 실행이라는 가장 위험·고빈도 영역에 효과를 집중시킨 결과입니다.

PROJECT 02

quantus-pipeline — 레거시 데이터 파이프라인 Airflow 현대화

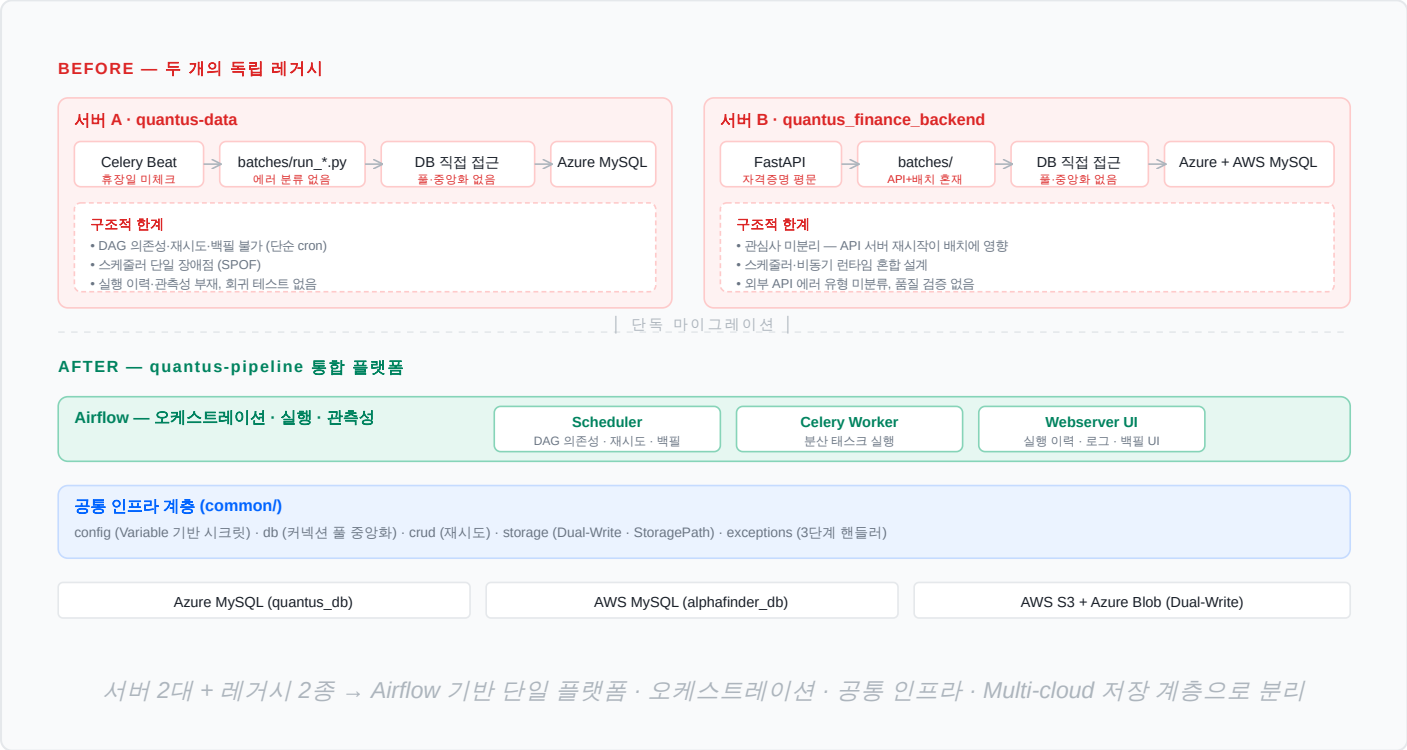
STOA Investment · 2026.03 프로덕션 전환 · 단독 수행

Apache Airflow | SQLAlchemy | Celery | Docker Compose | AWS S3 · Azure Blob | pytest

프로젝트 배경

3년간 직접 유지보수한 두 레거시 데이터 시스템(~14K LOC)에서 반복되는 이슈를 역추적해보면 개별 버그보다 **구조적 한계**(스케줄러 SPOF · 오케스트레이션 부재 · 관심사 미분리 · 관측성 부재)가 원인인 경우가 많았습니다. 2026년 3월 18시간 중단 사고를 기점으로 Airflow 기반 통합 플랫폼으로 단독 재설계·전환했고, 1개월 안정화를 거쳐 레거시 2종 모두 폐기했습니다.

Before / After 아키텍처



핵심 구조 문제

개별 버그가 아닌 **세 가지 구조적 문제**가 반복 장애의 공통 원인이었습니다.

구조 문제	실제 발생한 영향
오케스트레이션 부재 — Celery Beat는 단순 cron, 실행 이력·DAG 의존성·재시도·백필 모두 불가	18시간 중단 사고 발생, 선행 태스크 실패가 하위 태스크에 무작위 확산
관심사 미분리 — API 서버와 배치가 동일 프로세스 + 에러 처리 미분류	서버 재시작이 배치에 영향 · 모든 예외를 동일하게 catch해 원인 파악에 수시간
관측성·회귀 검증 부재 — 실행 이력 추적·자동 테스트 없음	신규 기능 추가·리팩토링 시 영향 범위 확인 불가

핵심 설계 판단

CELERY BEAT → APACHE AIRFLOW

단순 cron 방식으로는 DAG 의존성·재시도·백필·관측성 어느 하나도 제대로 다룰 수 없었고, 18시간 사고가 그 정도로 길어진 직접 원인은 **실행 이력·의존성 추적 부재로 복구 경로 자체가 불명확**했기 때문입니다. Airflow를 Managed 서비스가 아닌 **자체 운영**으로 가져간 것은 현재 규모에서 비용 효율이 훨씬 좋고, 운영 부담도 감당 가능한 수준이라고 봤기 때문입니다. K8s까지 가는 건 지금 규모에는 과잉이라 제외했습니다.

DB 연결 중앙화 · 커넥션 풀 도입

기존 구조는 매 호출마다 커넥션을 새로 만들어서 고부하 시 커넥션이 폭증할 위험이 있었고, 클라우드 MySQL의 idle timeout에도 대응하지 못했습니다. SQLAlchemy 기반 커넥션 풀로 **연결 관리를 한 곳으로 모으고** 유효성 검사를 자동화해서 두 문제를 동시에 풀었습니다. 버전은 Airflow 의존성에 맞추되, 장기적으로 갈아탈 경로가 남아 있는 조합으로 골랐습니다.

스토리지 DUAL-WRITE 점진 전환

DB는 Azure, 스토리지만 AWS에 있는 cross-cloud 구조라 지연과 비용이 모두 불리했습니다. 그렇다고 빅뱅으로 이관할 수는 없었는데, **실서비스가 기존 스토리지 경로에 직접 의존**하고 있었기 때문입니다. **양쪽 기록 → 읽기 전환 → 단일화**의 점진 이관으로 이행기 비용을 감수하는 대신 무중단 전환을 선택했습니다. 한쪽 쓰기 실패 시에는 **다른 쪽 쓰기를 계속 진행해 전파를 막고, 실패 기록을 중앙 예외 파이프라인으로 올려 후속 재동기화**하는 방식으로 이행기 정합성을 관리합니다.

금융 파이프라인 특화 설계 — 거래일 센서 · API 동시성 제어

일반 ETL과 달리 금융 데이터 파이프라인은 **거래일·외부 API Rate Limit**이라는 도메인 특수성을 설계에 명시적으로 반영해야 했습니다. **거래일 센서(ShortCircuit)**로 휴장일에는 하위 태스크 전체를 자동 스킵해 "휴장일에 KRX 응답 없음 → 에러 알림 폭주"가 재발하지 않게 했고, 증권사 API 처럼 **외부 동시성 제약이 있는 소스는 Airflow Pool로 동시 호출 수를 중앙에서 제한**해 DAG 간 경합 없이 쿼터를 관리합니다. 데이터 품질 방어는 별도 게이트로 분리했습니다(아래 insight 참조).

운영 안정화 관점

프로덕션 전환 직후 1개월간 올라온 이슈들은 크게 두 가지 패턴으로 수렴했습니다. 하나는 **외부 API 문서를 있는 그대로 믿었을 때 생기는 실제 동작과의 차이**, 다른 하나는 **모든 예외를 똑같이 받는 포괄적 에러 처리가 실패 원인을 가려버리는 문제**. 이후로는 신규 외부 데이터는 여러 샘플로 실제 값을 먼저 확인한 뒤 사용하고, 에러 핸들러는 실패 유형별로 분기해서 다루는 것을 기본 원칙으로 두고 있습니다.

대표 사례 · 외부 API 문서 신뢰의 한계

재무 데이터 필드 단위 불일치 — 실서비스 팩터 오염

증상 시가총액이 실제 대비 **1/10,000 수준**으로 계산되어, 이 수치를 사용하는 전체 밸류에이션 팩터가 실서비스에 잘못 공급되는 상태였습니다.

진단 API 공식 문서에 명시된 단위와 실제 반환값의 단위가 달랐습니다. 문서만 신뢰한 결과.

대응 여러 종목을 실데이터로 크로스체크해 실제 단위를 확인한 뒤 변환 로직을 수정. 이어서 같은 지표를 다른 소스에서도 얻을 수 있는 fallback 필드를 1순위로 올려 단일 소스 의존 구조를 걷어냈습니다.

→ 이후 모든 외부 데이터에 실데이터 크로스체크를 기본 절차로 포함

결과와 남은 한계

항목	BEFORE (레거시)	AFTER (QUANTUS-PIPELINE)
스케줄링 안정성	오케스트레이션 부재 (단일 인스턴스, 실행 이력 부재, 18h 사고 전례)	실행 이력 영속화 · 자동 재시도 · 백필 가능 (HA는 의도적 미도입 — 후술)
DAG 의존성 관리	없음 (순차 가정만 존재)	명시적 DAG + 부분 재실행 가능
관심사 분리	API 서버와 배치 프로세스 혼재	파이프라인 전용 플랫폼 분리
에러 처리	실패 유형 미분류, 원인 파악 수시간	유형별 핸들러 + 구조화 로깅
품질 보증	회귀 테스트 사실상 없음	CI 자동화 · 테스트 커버리지 확보
운영 도구	실행 이력 추적 불가, 백필 수동	UI 기반 이력·로그·백필

레거시 완전 폐기 | 프로덕션 안정 운영 | 10 DAG · KR/US 다중 자산군 커버

현재는 **CD(배포 자동화)를 의도적으로 도입하지 않고 수동 Docker Compose 배포를 유지**하고 있습니다. (CI는 도입 — 위 표 참조) 배포 빈도와 환경 복잡도가 아직 자동화의 초기 구축 비용을 정당화하는 지점에 오지 않았다고 판단했기 때문이고, 배포 빈도가 일정 임계를 넘거나 환경이 다중화되는 시점에 도입할 예정입니다. 이행기의 이중 스토리지 비용, 일부 라이브러리 버전 고정 등 나머지 부채도 **해결 조건과 시점을 정해둔 상태로만 수용**하는 원칙을 적용하고 있습니다.

데이터 레이크 & 실서비스 연동

S3/Blob은 기본 역할이 **파이프라인 산출물의 백업 저장소**이지만, 동시에 **여러 서비스 서버가 단일 소스에서 동일한 데이터를 읽어갈 수 있는 공용 조회 지점**으로도 활용합니다. 각 서비스가 별도 데이터 복제본을 가지면 **동기화 비용**과 **버전 드리프트**가 발생하는데, 백업용으로 어차피 유지해야 하는 저장소를 **읽기 경로로도 겸용**함으로써 인프라를 늘리지 않고 "하나의 진실 소스(single source of truth)"를 확보하는 선택이었습니다. 파일 전송 기반 소비자(liveback·workstation)는 **레거시 호환 목적**으로 유지 중이며, 중기적으로 직접 조회 구조로 흡수할 예정입니다.

StoragePath 레지스트리는 `price.adjusted`, `index.base`, `factor.eodhd` 등 **13개 논리 경로를 표준화**한 추상화 계층입니다. 경로를 논리 이름으로 분리해두면 **버킷 이관·환경 분리 시 코드 변경 없이 레지스트리만 갱신**하면 되고, 위 결정 박스에서 다룬 **Dual-Write 3단계 전환(기록→읽기 전환→단일화)**도 이 추상화를 전제로 가능했습니다. MySQL은 **메타데이터 전용**으로만 두어(지원 종목 · 제외 티커 · 에러 로그) 시장 데이터 자체는 파일로 관리해 OLTP와 분석 워크로드의 부하를 분리했습니다.

> **품질 게이트 발동 시나리오:** 팩터 생성 단계에서 신규 데이터가 직전 대비 **5% 이상 감소**하면 업로드를 자동 중단하고 백업 파일을 만들어 Slack으로 경고합니다. 실서비스가 잘못된 데이터로 오염되는 것을 사전에 차단하는 **최후의 방어선** 역할이며, EODHD 제외 티커는 주간 자동 리포트로 별도 추적됩니다.

플랫폼 고도화 로드맵 — 데이터 마켓 · 피처 스토어

현재 구조는 **여러 소비자에게 동일 파일을 제공하는 "데이터 공급자"** 역할에 최적화되어 있습니다. 소비자 수가 늘고 ML 기반 전략이 확장될수록 **재사용성·거버넌스·학습-추론 일관성**이 새로운 병목으로 부각될 것으로 보고 있으며, 두 축의 상용 수준 고도화를 단계적으로 진행할 예정입니다.

축	핵심 역할	도입 단계	재활용 가능한 현재 자산
데이터 마켓 Data Marketplace	데이터 셋을 **제품(Data as a Product)**으로 공개. 카탈로그·스키마·오너·SLA·lineage를 표준화하고, 소비자가 검색·구독·계약(Data Contract) 기반으로 안정적으로 소비	① 카탈로그 UI · 메타데이터 스키마 확장 ② 소비자 구독 / 접근 권한 모델 ③ Data Contract 기반 스키마 호환성 검증	StoragePath 레지스트리 → 논리 경로 체계 메타데이터 DB → 카탈로그 시드 품질 게이트 → SLA 정의 기반
피처 스토어 Feature Store	ML 피처의 **학습-추론 일관성**과 **팀 간 재사용** 보장. Offline/Online Store 분리로 대량 학습 조회와 저지연 추론 조회를 동시에 충족	① Feature Registry · 버전 관리 ② Online Store 도입 (저지연 추론) ③ Point-in-Time Join으로 데이터 누수 방지	팩터 파이프라인 → 피처 산출 자산 Dual-Write 구조 → Offline Store 기반 Airflow 스케줄 → 피처 생애주기 관리

> **도입 트리거와 원칙:** 두 축 모두 **현 규모에서는 오버엔지니어링**이므로 즉시 도입하지 않습니다. 대신 **도입 시점을 조건으로 정의**합니다 — 데이터 마켓은 **내·외부 소비자 수가 임계를 넘어 카탈로그 없이 거버넌스가 불가능해지는 시점**에, 피처 스토어는 **ML 전략이 여러 개로 확장되어 피처 중복·학습·추론 불일치 위험이 실측되는 시점**에 도입합니다. 지금 구조의 **StoragePath·메타 DB·품질 게이트**는 이 확장 시 그대로 상위 계층의 기반으로 흡수될 수 있도록 설계했습니다.

PROJECT 03

퀀터스 프라임 — 글로벌 올웨더 액티브 전략

STOA Investment · 2024 – 현재 · 서비스 기획·구현·전략 설계·운용 단독 담당

Portfolio Optimization | Regime Detection | Risk Overlay | Feature Engineering | Statistical Testing | Walk-Forward

가설 수립 및 문제 구체화

퀀터스 플랫폼은 개별 팩터·자산배분 전략을 유저가 직접 조합하는 B2C 퀀트 도구입니다. 하지만 **플랫폼 학습 곡선이 높아 라이트·신규 유저가 이탈하는 문제**가 있었습니다. 이를 해결할 방법으로 "퀀터스가 직접 설계한 전략을 3-클릭으로 구독"하는 **퀀터스 프라임**을 기획했고, 전략 라인업의 핵심축으로 **글로벌 올웨더 액티브**를 배치했습니다. 기관에서 정립된 올웨더 프레임워크는 개인 투자자가 바로 운용하기 어려워, **실전 운용 가능한 형태로 재설계**하는 것이 핵심 과제였습니다.

전략을 "해결할 기술 문제"가 아닌 **네 가지 연구 가설**로 구조화하고, 각 가설에 검증 방법과 **반증 조건**을 함께 정의했습니다. 가설이 기각되면 시스템 전체가 아니라 해당 부분만 수정하도록 모듈을 설계 단계에서 분리했습니다.

→ **H1. 경제 체제 분산이 구조적 알파의 원천이다** — 성장·인플레 4분면에 대한 리스크 배분을 엔진 중심에 둠. *반증 조건*: 단일 체제(예: 성장 편향) 가중 포트폴리오가 일관되게 초과 수익을 내면 체제 분산의 가치는 낮음

→ **H2. 최적화 결과의 품질은 입력 데이터(공분산)의 품질에 종속된다** — 자산 수가 많고 관측 기간이 제한된 환경에서는 공분산 행렬이 수치적으로 불안정해져, 최적화기가 같은 데이터에 대해서도 결과가 크게 달라지는 문제. *반증 조건*: 여러 추정 방식으로 만든 공분산을 바꿔 넣어도 최종 비중 차이가 거래비용 수준 이하라면 입력 품질의 영향은 제한적

→ **H3. 레짐은 확률이지 단일 상태가 아니다** — 시장의 대부분 시간은 분면 사이의 전이에 위치하므로 하드 분류는 경계 근처 신호 반전을 유발. *반증 조건*: 하드 분류와 확률 가중의 실현 성능 차이가 무시할 수준이면 확률 구조의 부가가치 없음

→ **H4. 투자위원회 뷰는 data-driven optimum의 prior다** — 꼬리·체제 전환에 대한 ex-ante 추정 오차를 Strategic Weights가 완화하는 anchor 역할. *반증 조건*: prior 가중이 0일 때가 모든 지표에서 우세하면 prior는 noise

구조적으로는 **사전 지식(투자위원회 뷰)과 데이터(공분산)와 상황 신호(레짐)를 결합해 하나의 비중 벡터로 수렴**시키는 구조이며, 이는 자산배분 분야의 대표적 프레임인 **Black-Litterman**과 설계 의도가 유사합니다. 세 가지 성격이 다른 정보원을 한 엔진 안에서 조합하는 것이 핵심이며, 각 정보원의 가중치는 상품 레벨별 파라미터로 조정됩니다.

전략 파이프라인 아키텍처

*주 파이프라인: 데이터 → 레짐·꼬리위험 피처 → 포트폴리오 최적화 → 리스크 오버레이 → 실전 집행 · **레짐 감지는 자산 가중치 입력에, 꼬리 위험은 오버레이 트리거에 반영** · 통계적 선행성 검증은 신규 피처 도입 시 게이트로 별도 운용*

핵심 설계 판단

포트폴리오 최적화 — 계층적 리스크 예산 구조

기존 Mean-Variance 방식은 공분산 추정 오류에 취약해서 **실전에서 소수 자산에 비중이 쏠리는 문제**가 있었고, 단순 Risk Parity는 자산군 간 균형과 자산군 내 효율성을 동시에 잡지 못했습니다. 이 두 문제를 분리해서 푸는 방향으로, **자산군 간 리스크 균형을 먼저 맞추고 자산군 내부에서 효율성을 조절**하는 2단계 구조로 재설계했습니다.

공분산 추정은 최적화의 입력 품질을 결정하는 핵심이라, 단일 연산이 아닌 **4단계 전처리 파이프라인**으로 구성했습니다. 각 단계는 이전 단계의 출력을 전제로 다음 단계가 동작하는 구조이며, 한 단계라도 생략하면 이후 단계의 가정이 깨집니다.

→ **EWM (지수가중 이동)** — 오래된 관측일수록 가중치를 줄여 최근 시장 상태를 반영. 단순 평균은 레짐 전환에 반응이 느려서, **시장 체제 변화를 시계열에 반영하기 위한 출발점**

→ **Ledoit-Wolf shrinkage** — 자산 수에 비해 관측 기간이 상대적으로 부족할 때 공분산이 **수치적으로 불안정**해지는 문제를, 안정적인 기준 행렬 쪽으로 보정해서 완화. 이 단계 없이는 이후 최적화 솔버가 자주 실패

→ **RMT (랜덤 행렬 이론) 노이즈 제거** — shrinkage가 대체하지 못하는 잔여 노이즈를 **랜덤 데이터에서 기대되는 수준의 신호를 노이즈로 간주해 제거**. 최종 비중이 노이즈에 덜 흔들리게 함

→ **PD 보정 (양정치 행렬 보정)** — 최적화기가 요구하는 수학적 조건을 만족시키기 위한 최종 가드. 극히 작은 수치 오차로 **솔버가 입력을 거부하는 경우를 방지**

임의 조합이 아니라 **각 단계가 서로 다른 문제를 해결**하며, 어느 것도 다른 것으로 대체되지 않는다는 것이 설계의 핵심입니다.

DUAL-AXIS 레짐 감지 — 해석 가능성 우선의 규칙 기반 설계

실험 단계에서 **비지도 학습 기반 클러스터링**(데이터로부터 자동으로 시장 상태를 군집화하는 ML 방식, HMM·GMM 등)도 구현해봤지만, **출력 클러스터에 경제적 의미를 사후적으로 붙여야 한다는 근본 한계**가 실서비스 신뢰성에서 문제가 됐습니다. 대신 경제적 의미(성장·인플레 축)가 **설계 단계에서 직접 반영되는 규칙 기반 레짐 감지**를 프로덕션에 올려, 해석 가능성과 운영 안정성을 우선하는 선택을 했습니다.

꼬리 위험 관리 — 복수 지표 통합

단일 지표에 의존하면 극단 국면에서 신호를 놓치는 경우가 많다는 교훈이 바탕입니다. 특히 **단순히 과거 분포만 보고 추정하는 방식은 꼬리(극단 손실 구간)를 과소추정**하는 한계가 있어, 성격이 다른 세 가지 지표를 결합하는 방향으로 설계했습니다. **변동성 동적 추정**은 현재 시장의 불안정성을, **극단값 분포 모델링**은 꼬리의 분포 자체를, **이상 수익률 탐지**는 과거 패턴에서 벗어난 움직임을 각각 포착합니다. **판단 오류의 상관관계가 낮은 세 신호를 통합**해 한 지표가 놓친 신호를 다른 지표가 포착하도록 하는 구조이며, 2022 고변동성 국면 전례가 이 다층 방어 설계의 직접적 동기였습니다.

시그널 상태 머신 — 라이브와 백테스트의 구조적 동형성

라이브 운용과 백테스트가 **같은 코드 경로를 공유**하는 것이 설계의 핵심 요구였습니다. 백테스트 결과가 아무리 좋아도 라이브 로직이 따로 돌면 실전 재현성이 깨지기 때문입니다. 세 가지 가중치 도메인을 명시적으로 분리했습니다.

- → **latest_target** — 매 시그널 평가 시 최신 최적화 산출물. 매일 갱신
- → **last_rebalanced** — 리밸런싱 임계(최대 비중 편차)를 통과해 실제로 채택된 가장 최근 비중. 임계 미통과 시 갱신되지 않음
- → **current** — 실 계정에 체결된 포지션 상태. 부분 체결·주문 실패가 반영되어 `last_rebalanced` 와 의도적으로 다를 수 있음

이 3-tier 분리로 "**최적화는 매일, 리밸런싱은 임계 통과 후, 체결은 또 다른 상태**"가 독립적으로 관리됩니다. 동일 시그널 생성 함수가 라이브(DB의 `last_rebalanced` 참조)와 walk-forward 백테스트(시뮬레이터의 이전 승인 비중 참조)에서 **동형으로 호출**되어, **백테스트가 라이브의 정확한 시뮬레이션**임을 구조적으로 보장합니다. 최종 주문 집행은 **Project 01의 분산 트레이딩 시스템**으로 위임되며, 전략과 집행이 수평 확장 가능한 형태로 분리됩니다.

데이터 기반 가정 검증 — 운용 중 기존 로직을 뒤집은 재설계

초기 올웨더 운용 중에는 **"매크로 지표가 악화되면 전량 매도"** 방식의 마켓타이밍이 돌고 있었는데, 실제로는 매도 직후 빠른 반등에 재진입을 놓치면서 손실만 반복됐습니다. 문제의 본질은 해당 지표의 시장 선행성이 **데이터로 검증된 적이 없다는 것**이었습니다. 인과성과 정보 전달을 측정하는 **3종의 독립적인 통계 검정을 교차 적용**한 결과, 단기 시계열에서 유의한 선행 관계가 확인되지 않았고, 이 근거로 기존 전량 매도 로직을 폐기하고 **수익곡선 기반 재진입 + 모멘텀 마켓타이밍**으로 재설계했습니다.

> 이 사례의 핵심은 "새 로직이 더 좋다"가 아니라 **"기존 로직이 왜 무효한가"를 먼저 데이터로 보여준 것**입니다.
> 재설계의 설득력은 여기서 나왔고, 이후 내부 가정을 건드릴 때마다 비슷한 절차를 재사용하고 있습니다.

상품-엔진 연결 — 단일 엔진의 레벨 스펙트럼

퀀터스 프라임은 공격성 수준별로 L1~L5 다섯 단계의 상품을 제공합니다. 이 스펙트럼은 별개의 전략이 아니라 **동일한 엔진 위에서 세 가지 파라미터 축을 단조 이동**시켜 구현했습니다.

→ **strategic_blend (β)** — Strategic Weights가 자산군 비중을 주도하는 정도. L1은 Risk Budget 엔진이 주도하고, L5는 prior의 신뢰도를 크게 두어 레짐 신호가 강하게 반영되는 구조

→ **sharpe_tilt_strength (α)** — 자산군 내부에서 Sharpe 정보를 반영하는 강도. L1일수록 순수 Risk Parity에, L5일수록 효율성 반영에 가까움

→ **regime_sensitivity (s)** — 레짐 틸트가 최종 비중에 미치는 영향력. 공격적 레벨일수록 레짐 신호의 가중이 커지도록 단조 증가

결과적으로 **L1은 올웨더 원형(Risk Parity 주도)에, L5는 Strategic Weights + 레짐 주도 구조에 가깝게** 동작합니다. 단일 코드·단일 유니버스·단일 신호 체계를 공유하면서 파라미터 조합만으로 전체 스펙트럼을 커버하는 구조라, 신규 레벨을 추가하거나 기존 레벨을 튜닝할 때 **엔진 재구현 없이 파라미터 커브만 조정**하면 됩니다. 다만 L1~L5는 같은 엔진·같은 신호를 공유하므로 **레벨 간 실현 수익률 상관이 높고 "레벨 분산 = 체제 분산"이 아니라는 점**은 상품 설계의 명시적 한계로 관리하고 있습니다.

운용 성과

지표	올웨더 액티브	벤치마크 (S&P 500)	결과
누적 수익률 (2024-12-20 ~ 2026-01-31)	**+16.8%**	+17.0%	벤치마크 수준
Sharpe Ratio	**1.5**	0.88	약 1.7배 우위
MDD (Max Drawdown)	**-8.0%**	-18.9%	절반 이하

단일 전략 AUM 50억 돌파 | **서비스 전체 매출 40% 기여** | Total AUM 500억

벤치마크와 유사한 수익률을 절반 이하의 낙폭으로 달성했다는 점이 올웨더의 본래 목적(방어적 자산배분)에 부합하는 결과입니다. Sharpe 기준으로는 약 1.7배 우위, MDD 기준으로는 벤치마크 대비 절반 이하로 하방 방어력이 뚜렷하게 드러났습니다.

운용 지속성 — 업데이트 주기와 알파 소멸 대응

퀀트 전략은 한 번 만들고 끝나는 것이 아니라, 시장 구조 변화에 따라 **지속적인 검증·재설계**가 전제되어야 합니다. 이를 위해 세 가지 절차를 기본 루틴으로 두고 있습니다.

→ **정기 모니터링** — 주 단위로 지표별 실전 성과를 벤치마크와 대비해 Sharpe·MDD·자산군 기여도 추이를 추적. 일정 수준 이상의 성과 저하가 지속되면 원인 분석 우선순위에 올림

→ **피처 유효성 재검증** — 포트폴리오 최적화에 사용 중인 피처도 **과거 한 번 검증했다고 영구 유효하다고 가정하지 않음**. 반기 단위로 하위 사례에서 다룬 통계적 선행성 검증을 재실행해 여전히 유의한지 확인

→ **커버리지 방어를 위한 자산군 확장 — 알파 소멸**(시간이 지나며 전략의 초과 수익이 줄어드는 현상)의 주 원인 중 하나는 시장 구조 변화입니다. 단일 자산군에 의존하는 취약점을 줄이기 위해 편입 자산군을 점진적으로 확장 중이며, 대체자산·디지털 자산 편입을 검토하는 단계

알려진 한계와 완화 장치

백테스트 성과가 좋게 나왔을 때 가장 경계해야 하는 것은 **시스템을 "항상 옳은 것"처럼 착각하는 일**입니다. 이를 구조적으로 방지하기 위해 시스템의 약점을 **수학·구조·운영 3축**으로 분류해 상시 관리하고, 이 전략이 **무엇을 주장하고 무엇을 주장하지 않는지**를 운용 문서에 명시하고 있습니다.

축	인지하고 있는 한계	완화 · 관리 방식
수학·통계 가정	레짐 감지는 성장·인플레 두 축을 독립으로 가정하지만, 실제 두 축의 상관은 0이 아니며 구간에 따라 부호도 바뀜	확률의 **상대 순위**는 보존되며 절대 수준만 편향. 틸트 영향력을 regime_sensitivity로 제어해 편향 확대를 억제
	CVaR 오버레이는 "현재 비중이 과거에 유지되었다"는 **가상의 가정**(실제로는 과거 시점마다 비중이 달랐음)으로 계산. 구현 단순화를 위한 근사	이 오버레이는 예측이 아닌 **방향성 소프트 축소**. 현재 비중과 최근 체제의 정합도를 감지하는 용도로 국한
구조·설계	대체자산(Alternatives) 카테고리가 사실상 **단일 자산** 선택에 의존. 자산군 내 분산 효과 부재	해당 자산 자체의 추세추종 내부 분산으로 부분 대체. 대체자산군 다변화는 커버리지 확장 로드맵의 일부
	거래비용·슬리피지가 백테스트에 미반영. gross 기준이라 실전 net 성능과 괴리 가능	리밸런싱 임계값을 과도한 거래 억제 방향으로 보수적 설정. turnover-net return 민감도 분석은 후속 과제
운영·감시	리밸런싱 임계값이 경험적 기준으로 설정되어 있고 데이터 기반 민감도 분석이 아직 없음	임계값 구간별 turnover(리밸런싱 빈도) vs **tracking error**(벤치마크 대비 편차) 민감도 측정을 모니터링 항목으로 편입하여 사후 검증 계획

이 전략이 주장하지 않는 것 — Risk Budget 해가 실현 리스크 기여도와 정확히 일치한다고 주장하지 않으며, 레짐 감지가 미래 수익률을 예측한다고 주장하지 않습니다. CVaR 오버레이가 실제 꼬리를 방어한다고 주장하지 않으며, 방향성 있는 노출도 축소일 뿐입니다. **한계를 명시적으로 선언하는 것이 백테스트 과장 방지의 첫 번째 장치**라고 보고 있습니다.